

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**NAME:A SHIVA SHANKAR REDDY**

**REG NO:192425174**

**COURSE NAME:DESIGN AND ANALYSIS OF ALGORITHMS**

**COURSE CODE:CSA06**

### **CO4\_AT1 — Coding Challenge Task**

#### **Question 1 — 0/1 Knapsack Optimization using Dynamic Programming**

*CO4: Solve optimization problems using dynamic programming and greedy strategies. (BL4, BL5)*

#### **1. Question (As Given)**

##### **Q1. 0/1 Knapsack Optimization**

A warehouse manager must load items into a truck that has a fixed carrying capacity. Each item has a specific weight and associated profit, and items cannot be split or reused. The manager needs to decide which combination of items should be selected so that the total profit is maximized without exceeding the weight limit. Use Dynamic Programming to determine the optimal profit.

##### **Input Format:**

n = number of items

Values[] = profit of items

Weights[] = weight of items

W = capacity

##### **Sample Input:**

n = 3

Values = 60 100 120

Weights = 10 20 30

W = 50

##### **Expected Output:**

Maximum Profit = 220

## 2. Approach / Algorithm

The 0/1 Knapsack problem is solved using Dynamic Programming (bottom-up tabulation), which avoids the exponential time of brute-force recursion by storing solutions to overlapping sub-problems.

### Steps of the Algorithm:

- Create a 2D DP table  $dp[i][w]$ , where  $i$  ranges over items  $(0..n)$  and  $w$  ranges over capacities  $(0..W)$ .
- $dp[i][w]$  represents the maximum profit achievable using the first  $i$  items with a knapsack of capacity  $w$ .
- Base Case:  $dp[0][w] = 0$  for all  $w$ , and  $dp[i][0] = 0$  for all  $i$  (no items or no capacity means zero profit).
- Recurrence Relation: If  $weights[i-1] \leq w$ , then  $dp[i][w] = \max(values[i-1] + dp[i-1][w - weights[i-1]], dp[i-1][w])$ ; otherwise  $dp[i][w] = dp[i-1][w]$ .
- The final answer, the maximum achievable profit, is stored in  $dp[n][W]$ .
- A backtracking pass over the DP table identifies which items were actually included in the optimal solution.

**Time Complexity:**  $O(n \times W)$

**Space Complexity:**  $O(n \times W)$  (can be optimized to  $O(W)$  using a 1-D array)

### 3. Source Code

The program below (knapsack.py) implements the 0/1 Knapsack algorithm using Dynamic Programming in Python.

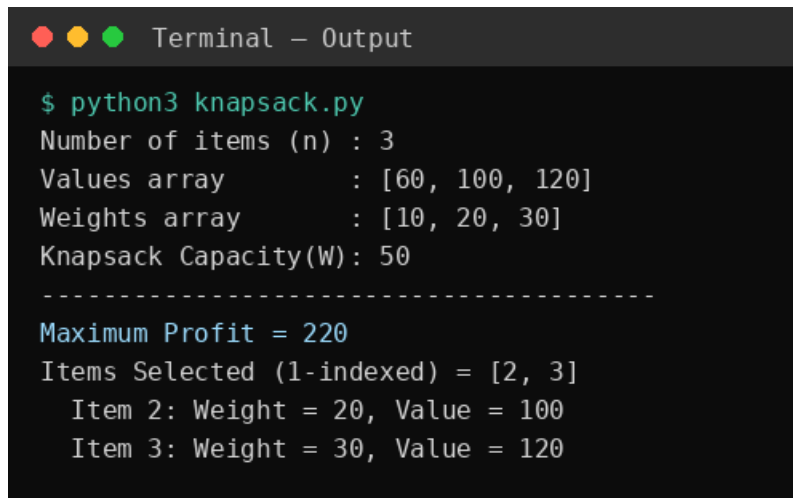
```
knapsack.py

1  """
2  C04 : Solve optimization problems using dynamic programming and greedy strategies.
3  Q1. 0/1 Knapsack Optimization using Dynamic Programming
4  """
5
6  def knapsack_01(n, values, weights, W):
7      # dp[i][w] = maximum profit using first i items with capacity w
8      dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
9
10     for i in range(1, n + 1):
11         for w in range(0, W + 1):
12             if weights[i - 1] <= w:
13                 dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
14                               dp[i - 1][w])
15             else:
16                 dp[i][w] = dp[i - 1][w]
17
18     # Trace back to find selected items
19     selected_items = []
20     w = W
21     for i in range(n, 0, -1):
22         if dp[i][w] != dp[i - 1][w]:
23             selected_items.append(i)
24             w -= weights[i - 1]
25     selected_items.reverse()
26
27     return dp[n][W], selected_items
28
29
30 if __name__ == "__main__":
31     n = 3
32     values = [60, 100, 120]
33     weights = [10, 20, 30]
34     W = 50
35
36     print("Number of items (n) :", n)
37     print("Values array      :", values)
38     print("Weights array     :", weights)
39     print("Knapsack Capacity(W):", W)
40     print("-" * 40)
41
42     max_profit, chosen = knapsack_01(n, values, weights, W)
43
44     print("Maximum Profit =", max_profit)
45     print("Items Selected (1-indexed) =", chosen)
46     for idx in chosen:
47         print(f"Item {idx}: Weight = {weights[idx-1]}, Value = {values[idx-1]}")
```

*Fig 1: Source Code Screenshot — knapsack.py*

## 4. Output / Execution Screenshot

The program was executed using the sample input ( $n = 3$ , Values = [60, 100, 120], Weights = [10, 20, 30],  $W = 50$ ). The output obtained is shown below.



```
Terminal - Output

$ python3 knapsack.py
Number of items (n) : 3
Values array       : [60, 100, 120]
Weights array      : [10, 20, 30]
Knapsack Capacity(W): 50
-----
Maximum Profit = 220
Items Selected (1-indexed) = [2, 3]
  Item 2: Weight = 20, Value = 100
  Item 3: Weight = 30, Value = 120
```

Fig 2: Program Output — Maximum Profit = 220

The program's output matches the expected output given in the question: Maximum Profit = 220, achieved by selecting Item 2 (weight 20, value 100) and Item 3 (weight 30, value 120), for a total weight of 50 and total profit of 220.

## 5. Analysis — DP Table Trace

The table below shows the computed  $dp[i][w]$  values (maximum profit) for selected capacities, built row by row as each item is considered:

| i \ W  | 0 | 10 | 20  | 30  | 40  | 50  |
|--------|---|----|-----|-----|-----|-----|
| Item 0 | 0 | 0  | 0   | 0   | 0   | 0   |
| Item 1 | 0 | 60 | 60  | 60  | 60  | 60  |
| Item 2 | 0 | 60 | 100 | 160 | 160 | 160 |
| Item 3 | 0 | 60 | 100 | 160 | 180 | 220 |

Observation: The optimal value  $dp[3][50] = 220$  is obtained. Backtracking from  $dp[3][50]$ , since  $dp[3][50] \neq dp[2][50]$  (160), Item 3 (weight 30, value 120) is included; moving to  $dp[2][20]$ , since  $dp[2][20] \neq dp[1][20]$  (60), Item 2 (weight 20, value 100) is included; the remaining capacity is 0, so Item 1 is excluded. This confirms the optimal selection {Item 2, Item 3} with total profit 220.

### Comparison with Greedy Strategy:

A greedy approach (choosing items by highest value/weight ratio first) does not guarantee an optimal solution for the 0/1 Knapsack problem, because items cannot be fractionally taken. For this reason, Dynamic Programming — which explores all valid combinations implicitly via the DP table — is the correct and provably optimal strategy for 0/1 Knapsack, whereas the greedy strategy is only optimal for the Fractional Knapsack variant.

## **6. Conclusion**

The 0/1 Knapsack Optimization problem was successfully solved using the Dynamic Programming (tabulation) approach. The program correctly computed the maximum achievable profit of 220 for the given sample input, matching the expected output. The DP table trace confirms that the optimal item combination is Item 2 and Item 3, with a combined weight of 50 (within the capacity limit) and a combined profit of 220. This demonstrates that Dynamic Programming guarantees an optimal solution for the 0/1 Knapsack problem by systematically evaluating all include/exclude decisions for each item, unlike a greedy strategy which may fail to produce the optimal result in this variant of the problem.